AI PRODUCT PROTOTYPE · PROJECT SPECIFICATION

# TasteTrace

## Explainable, diversity-aware music discovery

A functional web application that combines hybrid recommendation, long-tail artist discovery, diversity reranking, and traceable review evidence.

**CORE PROPOSITION**

Help listeners find music they are likely to value without repeating the same artists, over-rewarding popularity, or hiding the recommendation logic.

**Prepared for Imperial College London**

Big Data, AI & Machine Learning · Group Project

Version 1.0 · June 2026

01 · EXECUTIVE SUMMARY

# Product and business case

TasteTrace is a public-ready prototype for listeners who want more adventurous and accountable music discovery. The product starts with a small set of songs the user already values, converts those songs into a transparent taste representation, ranks a broad catalogue, and reranks the final set to reduce artist and album repetition.

**BUSINESS PROBLEM**

Existing recommendation feeds reduce search effort but often create repetition, popularity bias, weak explainability, and limited visibility for growing artists.

## How the prototype satisfies the assignment

| Requirement | TasteTrace evidence |
|---|---|
| Real business problem | Selection overload, recommendation repetition, and weak trust. |
| Functional software | Interactive React application with five working views. |
| AI component | Natural-language preference interpretation and grounded explanations. |
| ML component | Feature similarity, hybrid ranking, feedback, and diversity reranking. |
| Data component | Structured catalogue, evidence metadata, source links, and scalable architecture. |
| Product management | Clear users, scope, risks, success measures, and release sequence. |

## Prototype status

- Credential-free and reproducible.
- More than 80 curated tracks and more than 70 distinct artists.
- One-artist-per-result-set constraint in the default recommendation set.
- Outbound RYM, AOTY, and Pitchfork searches without copied live scores.
- Automated tests, production build, and zero known npm audit vulnerabilities.

02 · USER PROBLEM

# Why a different discovery experience is useful

| Pain point | Observed product pattern | TasteTrace response |
|---|---|---|
| Repetition | Same artist, album, or scene dominates the set. | MMR reranking plus hard artist and album penalties. |
| Popularity loop | Safe, familiar catalogue receives disproportionate exposure. | Long-tail value and emerging-artist priority. |
| Black-box ranking | Users see a result but not the evidence. | Seven visible component scores and bridge seeds. |
| Fragmented context | Recommendation and critical context live separately. | Direct review-source searches on every result. |
| Flat taste labels | A user becomes one genre or mood. | Multi-dimensional taste profile and controlled blind spot. |

## Target users

- Curious listeners who want a faster route into unfamiliar music.
- Heavy listeners who already understand genres and want deeper long-tail discovery.
- Users who value editorial or community evidence before committing listening time.
- Music platforms seeking a trust, diversity, or catalogue-coverage feature.
- Independent and growing artists who benefit from more balanced exposure.

## Value hypothesis

A more transparent and diverse recommendation set should increase useful discovery, catalogue coverage, saves, session depth, and trust. These are hypotheses for future evaluation, not claimed results.

03 · PRODUCT SEQUENCE

# End-to-end user journey

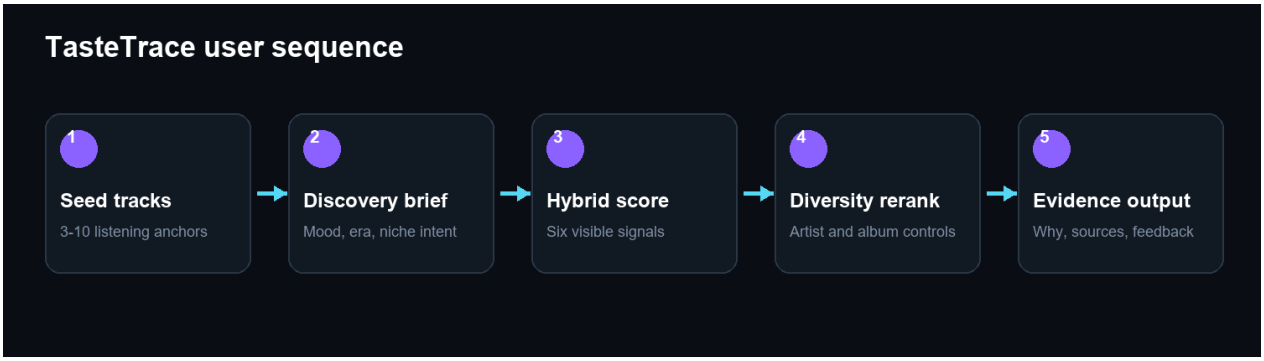


*Figure 1. The complete interactive recommendation sequence.*

## Working screens

| View | Primary job |
|---|---|
| Discover | Select seeds, write a brief, control exploration and inspect ranked tracks. |
| Artist Radar | Turn matching tracks into niche and growing artist discovery. |
| Taste Profile | Explain stable preferences, feature centre, and underexplored directions. |
| Evidence | Trace each result to model evidence and external review searches. |
| Model Lab | Adjust weights and observe relevance-diversity trade-offs. |

## Feedback loop

Like, dislike, and save actions persist in the browser. They immediately adjust the deterministic ranking, demonstrating the interaction contract for a later learning-to-rank system.

04 · SYSTEM ARCHITECTURE

# Current and production architecture

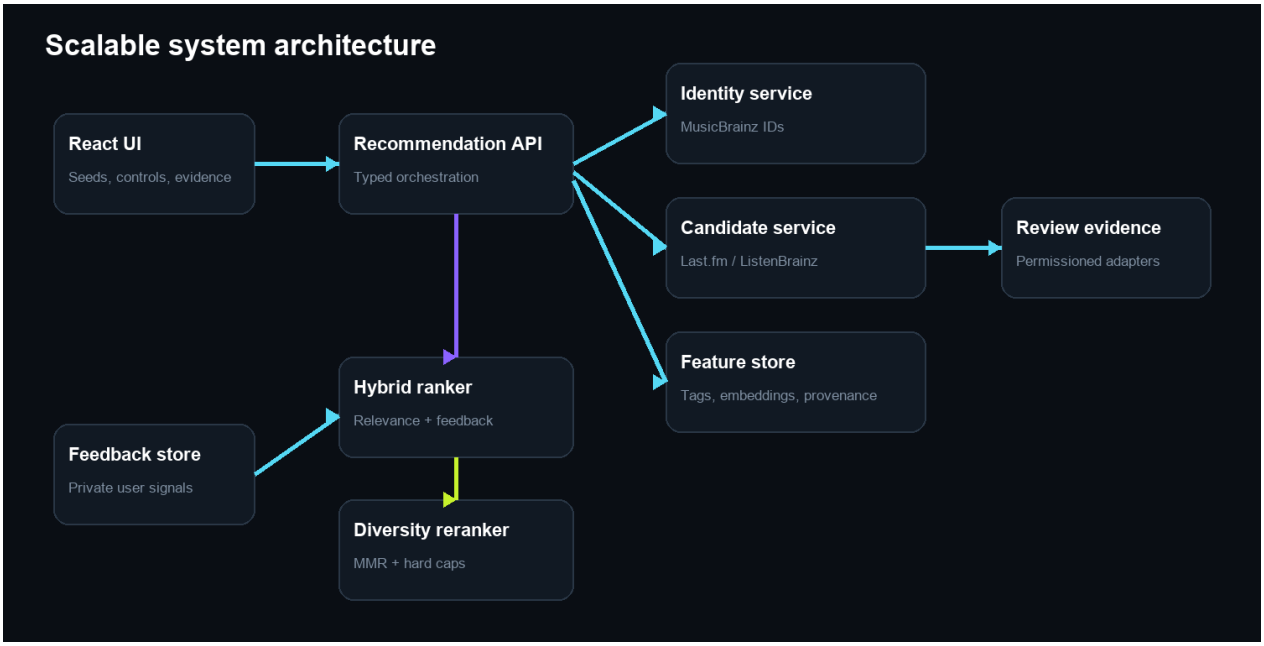


*Figure 2. Target architecture with independent identity, candidate, feature, ranking, evidence, and feedback services.*

## Current prototype

The implemented version runs entirely in the browser. This makes the classroom and public demo reliable, prevents credential leakage, and keeps every recommendation reproducible.

## Production transition

1. Use MusicBrainz recording and release-group IDs as canonical keys.
2. Add cached Last.fm and ListenBrainz candidate adapters.
3. Move features and provenance into a queryable store.
4. Version the ranker and reranker independently.
5. Store feedback only with explicit privacy controls.

05 · RECOMMENDATION MODEL

# Hybrid ranking and diversity reranking

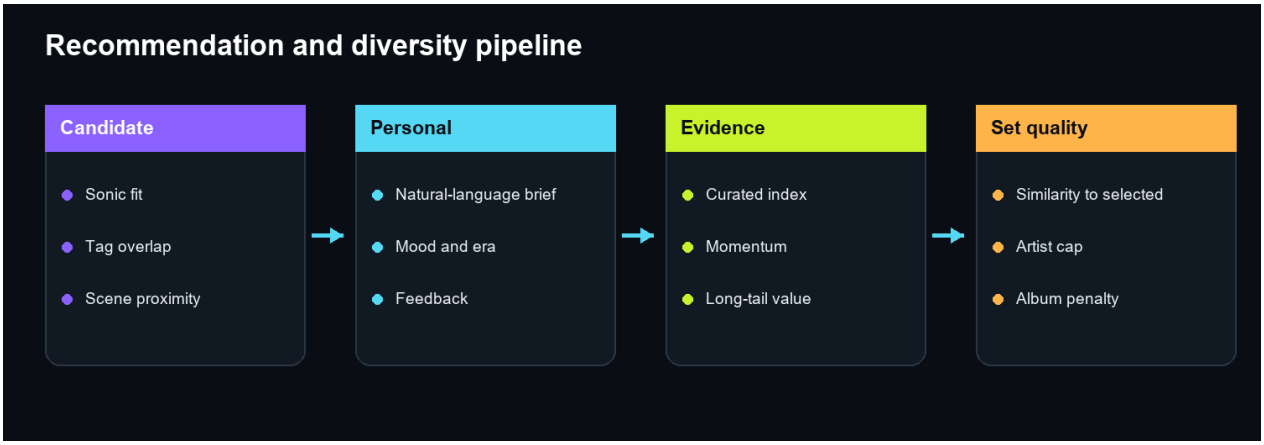


*Figure 3. Relevance and set quality are separate stages.*

## Base formula

LIKE INDEX

30% sonic fit + 20% tags + 14% scene + 12% brief + 10% evidence + 14% discovery - popularity penalty + feedback adjustment.

## Diversity stage

The highest-scoring track is selected first. Each subsequent candidate loses score when it resembles tracks already selected. Same-artist and same-album penalties are stronger than ordinary redundancy, while an emerging-artist bonus improves coverage when the set has none.

## Interpretation

- The 0-100 output is a fit index, not a probability.
- Evidence strength is not equivalent to personal relevance.
- Momentum is displayed separately from model fit.
- A lower-ranked candidate may improve the set by adding a new artist or scene.

06 · CATALOGUE AND EVIDENCE

# Data design and source governance

| Data group | Prototype implementation | Production replacement |
|---|---|---|
| Identity | Curated title, artist, album, year, country. | MusicBrainz IDs and confidence-aware resolution. |
| Tags | Curated genre, mood, and scene labels. | Permissioned tags plus multilingual curation. |
| Audio style | Deterministically derived feature values. | Licensed features or local audio embeddings. |
| Popularity | Prototype long-tail feature. | Dated, source-specific popularity measures. |
| Momentum | Prototype emerging-artist feature. | Time-windowed growth with provenance. |
| Reviews | Outbound RYM, AOTY, Pitchfork searches. | Licensed adapters with attribution and timestamps. |

## Why live review scores are not copied

Review websites have different score definitions, access rules, and update schedules. Copying visible values without a stable authorised interface would create legal, technical, and provenance risks. The prototype therefore keeps its own clearly labelled evidence index separate and directs users to the original source.

## Big Data position

The bundled catalogue is not presented as big data. The project demonstrates how high-volume metadata, listening events, embeddings, reviews, and feedback would be joined and governed in a production pipeline.

07 · PRODUCT DIFFERENTIATION

# What is meaningfully different

| Capability | Typical feed | TasteTrace |
|---|---|---|
| Recommendation logic | Hidden or minimally explained. | Component scores and bridge evidence. |
| Diversity | Often implicit. | Visible control and explicit reranking. |
| Artist discovery | Track-first and popularity-heavy. | Artist Radar with fit, tier, and momentum. |
| Review context | Separate browsing task. | Direct source trail for each album. |
| Taste reflection | Genre summaries. | Feature centre plus controlled blind spot. |
| Model experimentation | Not user-accessible. | Interactive Model Lab. |

## Design language

The interface adopts a dark research-console aesthetic. Violet indicates model state, lime indicates discovery, cyan indicates evidence, and semantic colours are reserved for feedback. Dense information hierarchy supports serious comparison without becoming a dashboard of decorative cards.

## Accessibility and responsiveness

- Keyboard focus states on interactive controls.
- Text labels for icon-only actions.
- Responsive navigation and stacked mobile layouts.
- No dependence on album artwork for identity or meaning.
- Colour is supported by labels and numeric values.

08 · EVALUATION

# How the product should be tested

| Evaluation layer | Metrics or checks |
|---|---|
| Functional | Search, seed selection, settings, feedback, navigation, review links. |
| Relevance | Precision@k, recall@k, NDCG, saves, rejection rate. |
| Diversity | Intra-list diversity, unique artists, unique albums, catalogue coverage. |
| Discovery | Novelty, popularity distribution, emerging-artist exposure. |
| Trust | Explanation usefulness, correction behaviour, source-link use. |
| Reliability | Tests, build, audit, API fallback, deterministic fixture. |
| Fairness | Exposure by region, language, audience tier, and catalogue segment. |

## Current verification

- Four automated recommendation-engine tests pass.
- TypeScript production build passes.
- npm dependency audit reports zero known vulnerabilities.
- Final sets contain distinct artists under default constraints.
- Scores remain bounded and review links are generated.

**NO UNSUPPORTED RESULTS**

No user study, online experiment, or production accuracy result has been conducted. The documentation therefore presents evaluation plans rather than invented outcomes.

09 · RESPONSIBLE AI

# Privacy, representation, and artist exposure

## Preference privacy

Music preference can correlate with culture, language, mood, religion, age, and community identity. The prototype stores only local feedback. A production service must provide consent, deletion, export, retention controls, and a prohibition on sensitive-trait inference.

## Catalogue representation

The expanded catalogue covers more countries and scenes than the original trial, but it remains incomplete and curator-dependent. Audience tiers and tags are subjective. Future work requires multilingual curation and exposure audits.

## Artist exposure

Long-tail bonuses and diversity constraints can improve exposure, but they can also be manipulated or create arbitrary thresholds. Momentum must be time-windowed, source-dated, and separated from personal fit.

## Human agency

- Users can inspect why a result appears.
- Users can change exploration and diversity.
- Users can reject or save results.
- Users can inspect independent external sources.
- The system avoids objective-quality language.

10 · BUILD SEQUENCE

# From prototype to open-source product

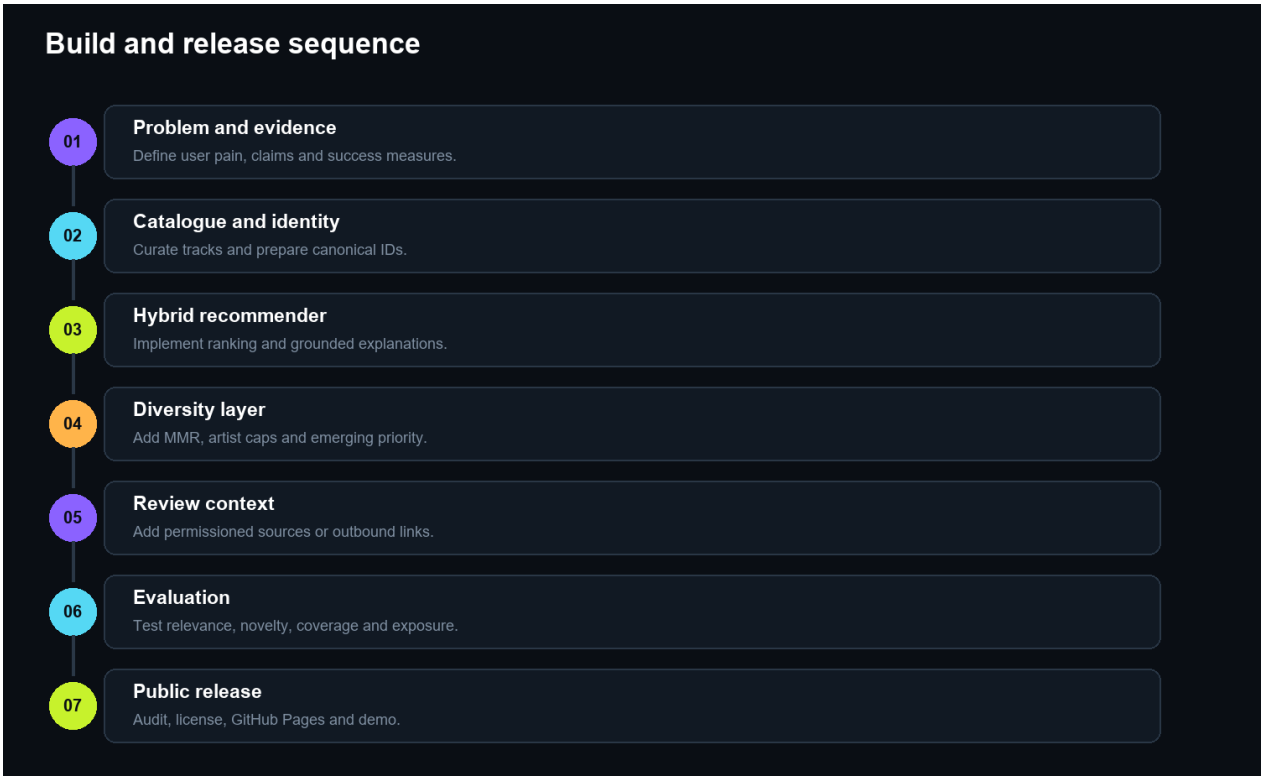


*Figure 4. Controlled implementation and release order.*

The prompt pack in the repository mirrors this sequence. Each phase has an objective, constraints, verification, and completion condition so future AI-assisted coding remains auditable.

11 · DELIVERY AND ROADMAP

# Current deliverables

- Working responsive web application.
- Expanded curated catalogue.
- Hybrid recommendation and diversity model.
- Artist Radar and review evidence workflow.
- English README and technical documentation.
- Structured AI coding prompt pack.
- Automated tests and production build.
- This visual project specification.

## Next release phase

6. Choose an open-source licence.
7. Initialise Git and review generated artefacts.
8. Add GitHub repository metadata and contribution guidance.
9. Configure GitHub Actions for tests and build.
10. Publish the static build through GitHub Pages.
11. Run the documented demo against the public URL.

## Presentation order

Problem -> seed and brief -> recommendation evidence -> diversity constraint -> Artist Radar -> review trail -> Taste Profile -> Model Lab -> limitations -> roadmap.

FINAL MESSAGE

TasteTrace does not try to replace human taste or criticism. It makes machine discovery more inspectable, more diverse, and easier to challenge.

APPENDIX

# Key source and extension references

| Reference | Use |
|---|---|
| MusicBrainz | https://musicbrainz.org/doc/MusicBrainz_API |
| ListenBrainz | https://listenbrainz.readthedocs.io/ |
| Last.fm API | https://www.last.fm/api |
| Rate Your Music | https://rateyourmusic.com/ |
| Album of the Year | https://www.albumoftheyear.org/ |
| Pitchfork | https://pitchfork.com/ |
| Maximal marginal relevance | Carbonell and Goldstein (1998), diversification of information retrieval. |

## Repository commands

LOCAL VERIFICATION

cd app | npm install | npm test | npm run build | npm audit

*Prepared from the implemented repository state. Public GitHub publication is intentionally reserved for the next project conversation.*